Temporal Exploit Prediction

Predicting *when* a CVE becomes weaponized — a survival-analysis modeling layer

Project report & technical overview · 2026-06-23

The one sentence to take away: the in-the-wild prediction ceiling is **DATA-limited, not model-limited**. Across the full survival toolbox (Cox, Random Survival Forest, gradient boosting, XGBoost-AFT, mixture-cure, DeepSurv/DeepHit) **nothing beats a penalized Cox** at the ~250–400 confirmed in-wild events available — so the achievable win is more / earlier *labels*, which is exactly what the VulnCheck wiring delivered.

This document consolidates two things: the **project report with figures**, and a **technical overview of the pipeline flow and feature set** (sections 3–5 and the appendices). Every headline number names its source artifact (`artifacts/*.json`) or living doc (`docs/*.md`) so it can be re-checked.

1. Executive Summary

This project is a **survival-analysis modeling layer** that predicts **when** a published CVE becomes publicly weaponized — a time-to-event problem — built on a pre-extracted, multi-source CVE timeline dataset (~338,000 CVEs). It is deliberately positioned as a **complement** to EPSS [1, 2], not a competitor: EPSS answers "will this be exploited in the wild in the next 30 days?" as a fixed-window binary; this project characterizes the upstream *weaponization pipeline timing* (PoC → Metasploit / Nuclei → KEV / 0-day) and produces a calibrated time-to-event curve EPSS structurally cannot give.

What was built.

- A leakage-safe dataset builder (four label sets: first-weaponization, per-signal, competing-risks, in-wild) over nine immutable handover parquets, with a feature-provenance audit trail and a content-hashed manifest.
- Models spanning the survival toolbox: Kaplan-Meier [3], penalized Cox PH [4, 5], Random Survival Forest [6], GPU XGBoost-AFT [7, 8], mixture-cure [9], DeepSurv/DeepHit [10, 11], plus a competing-risks / Aalen-Johansen core [12, 13].
- Rare-event-appropriate evaluation: IPCW c-index with bootstrap CIs, time-dependent AUC(t), PR-AUC, Brier/IPA, decision-curve net-benefit, and a rolling-origin (walk-forward) backtest.
- A **causal layer** (adjusted Cox + stabilized IPW + E-values) characterizing what *accelerates* weaponization, and a **patch-vs-exploit race** analysis.
- Live-fetch connectors for every documented source (CISA KEV, EPSS, NVD, Exploit-DB, PoC, Nuclei, Metasploit, Project Zero, VulnCheck KEV, MSRC, Shadowserver) and a merge layer.
- **337 tests** collected and green; memory held inside the ≤6–8 GB budget on every path.

The single most important finding.

The in-the-wild head is **data-limited**. On ~251–396 confirmed in-wild events the Cox model **ranks well** (c-index / AUC@90 ≈ 0.82–0.85) but its absolute probabilities **do not beat the base-rate null** (IPA ≈ 0) — and no fancier model improves on this. By contrast the first-weaponization head trains on **45,947 events**: there calibration works (IPA@180 = **+0.291**) but ranking is weak (c-index ≈ **0.60**) because the target is mostly PoC/disclosure logistics. **Two heads, two different limits — neither is the model.**

2. The Problem & Framing

The question. After a CVE is published, *when* does public exploitation capability appear? The clock origin is `cve_corpus.published`; the event is the earliest dated signal across five sources; CVEs with no signal are right-censored at the snapshot date. This is classic right-censored survival analysis, and the framing is itself a contribution: the field overwhelmingly treats exploitation as repeated 30-day binary classification (EPSS) [14, 15, 16], while explicit time-to-event modeling of weaponization is under-explored [17, 18].

CRITICAL framing caveat. Of all observed first-weaponization events, **~97% are public-PoC dates**. Only CISA KEV (`kev_date_added`) [19] and Google Project Zero 0-day are true in-the-wild signals — historically a few hundred events. **Any model trained on first-weaponization labels predicts time-to-public-tooling, NOT in-the-wild exploitation.** Keeping these two targets distinct is the backbone of every honest claim in this report.

The three distinct prediction targets, and why each exists.

- **First-weaponization** (45,947 events) — earliest of PoC / Metasploit / Nuclei / KEV / 0-day. Abundant, so it powers calibration and is where ML models can be exercised; but the target is dominated by PoC logistics, so ranking is intrinsically weak.
- **In-wild** (KEV + Google 0-day + VulnCheck; PoC excluded) — the project's meaningful target: actual exploitation, not tooling availability. Rare (251–396 events before VulnCheck), so it is the head whose ceiling is data-limited.
- **PoC → downstream transition heads** — reusable machinery to model the conditional escalation from a public PoC to genuine tooling (Metasploit / KEV), the deployable triage signal.

3. Data & Architecture (the Flow)

Two strictly separated layers — a deliberate reproducibility decision so modeling changes can never corrupt the source material:

- **Immutable handover / extraction** — the VRS-MongoDB extraction + enrichment scripts that produced nine parquet files. Read-only reproducibility material.
- **The modeling package** (`src/temporal_exploit/`) — the code developed here. It reads the handover parquets, *never* writes to them; all outputs go to a gitignored `artifacts/` dir.

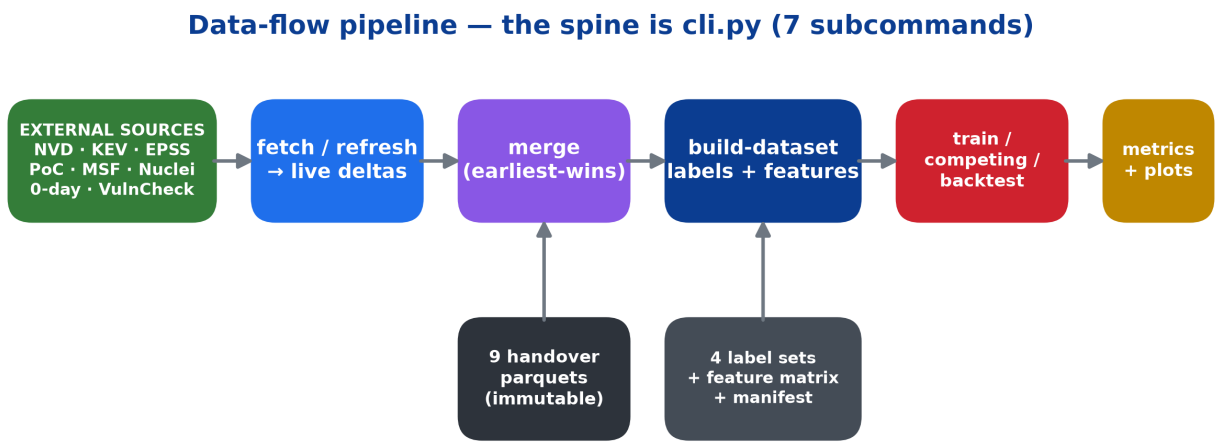

Figure 1. The data-flow pipeline. The spine is cli.py with seven subcommands; build-dataset is the integration hub that emits four label sets, the feature matrix, and a content-hashed manifest. Source: docs/architecture_flow.md.

The nine handover sources (all wired into the dataset):

Source	Parquet	Role
CVE corpus (NVD/VulnCheck)	cve_corpus	Features + clock origin (published)
PoC (Trickest + Nomi-sec)	poc_dates	Event source (~97% of first-weap events)
Metasploit	metasploit_dates	Event source / per-signal target
Nuclei	nuclei_dates	Event source / per-signal target

CISA KEV	kev_events	Event source + primary in-wild label
Google 0-day	google_0day	Event source + in-wild label
EPSS history (375M rows)	epss_history	EPSS-at-publication feature (leakage-safe)
ATT&CK chain	technique_cwe_chain	Tactic/technique one-hot features
VRS presence	vrs_presence	Snapshot-only, leakage-flagged (kept separate)

The **EPSS history is 375M rows / 3.7 GB** (one row group per day). It is read by streaming `iter_batches` + a fixed-size per-CVE numpy reduction (~0.6 GB peak) — a hard-won decision: an earlier pyarrow `isin(cve_ids)` pushdown retained ~5.8 GB and breached the memory budget. The only safe pushdown is by *date*.

4. Leakage-Safety Decisions (and Why Each Matters)

These are non-negotiable; they are *how the results stay valid*. Each excludes a concrete failure mode that would silently inflate apparent performance.

- **Exclude the CVE description text.** NVD *back-edits* descriptions after the event with phrases like "actively exploited" / "CISA" / "KEV" — pure temporal leakage. The masked, freshness-gated NLP path exists but is opt-in and off by default.
- **Exclude snapshot-time presence flags** (`vrs_presence`): whether a source *eventually* listed the CVE is knowledge from the future — a near-perfect leak. Kept in a separate, leakage-flagged parquet.
- **Exclude snapshot-time EPSS.** Today's EPSS for an old CVE leaks the future (a recent empirical re-evaluation quantifies the look-ahead: 2023 efficiency collapses 60.9% → 11.1% when scored honestly [20]). Only the *first EPSS reading after publication* is used.
- **Time-based splits, never random K-fold.** A model is only ever evaluated on CVEs published later than those it trained on.
- **The feature_provenance() audit trail.** Every emitted feature family gets a row with its source column and a leakage_status — adding a feature requires justifying its status.
- **Timezone discipline.** All date columns are `timestamp[ns, tz=UTC]`; mixing tz-aware and naive raises on subtraction — surfacing the bug rather than producing wrong durations.

A real near-miss this discipline caught: wrong schema column names (`cvss_v3_base_score` vs `cvss_v3_base`) silently produced *all-zero features* before they were caught — which is why the builder now fails loudly on missing required columns.

5. Feature Catalog

Features are grouped by **leakage status**. Only the publication-time-safe family is on by default; landmark and transition families are opt-in and require a matching clock restart; the snapshot family is deliberately excluded from the safe matrix (kept only for leakage audits). Source: `artifacts/feature_provenance.csv` (37 families).

Family	Examples	Status / when usable

Severity & structure	cvss_v3_base (+missing), severity one-hots, CVSS v3 vector components (AV/AC/PR/UI/S/C/I/A)	publication-safe (default)
Weakness & surface	cwe_* one-hots, weakness_count, has_weakness, vendor_count, product_count	publication-safe (default)
ATT&CK chain	has_attack_chain_mapping, attack_technique_count, attack_parent_* (CWE→CAPEC→ATT&CK)	publication-safe (default)
EPSS at publication	epss_at_publication, epss_percentile_at_publication, epss_at_publication_missing	publication-safe (first reading ≥ published)
Landmark (L=7/30d)	poc/metasploit/nuclei by/count/lag, epss_at_landmark (+pct/missing)	landmark-safe — only with restart_clock(L)
PoC→tooling transition	transition labels + counts for PoC→{MSF, Nuclei, KEV}	transition-safe (post-PoC clock)
Snapshot (EXCLUDED)	vrs_presence flags, snapshot EPSS, back-edited description text	leakage-flagged — NOT in the safe matrix

The point of the catalog: a feature is admissible only if its value is knowable at the moment the clock starts. “Publication-safe” means knowable at disclosure; “landmark-safe” means knowable at disclosure + L days *and* the model clock has been restarted to that landmark. Pairing a landmark feature with an unshifted clock would leak post-publication time — so the provenance status and the clock are checked together.

6. Models & Why

The model question was settled empirically (a prospective bake-off) and against the literature, not by preference: **penalized Cox is the in-wild backbone; everything fancier is a challenger that must earn its place prospectively, and so far none has.**

6.1 The classical backbone

- **Kaplan-Meier** — the unconditional reference and the null any model must beat.
- **Cox PH (penalized, lifelines [21])** — the in-wild backbone. Ridge shrinkage handles the borderline events-per-variable (~9.5 EPV [22, 23]); the penalizer is scaled by the event rate (undoing ~24× over-shrinkage on the rare target) with escalation on non-convergence.

6.2 The challengers, and why Cox won for in-wild

Model	AUC@90 (mean)	sd	recall@top-10%	IPA@90	fit time
Cox (penalized)	0.817	0.069	0.51	-0.003	78 s
Random Survival Forest	0.770	0.117	0.42	-0.016	384 s
Gradient-boosted (sksurv)	0.710	0.093	0.40	-0.001	362 s*
Mixture-cure	overturned	—	harmful	-0.27 @180d	—

* sksurv Cox-loss GBM [24, 25] is $O(n^2)$ per tree — only tractable after subsampling, which discards scarce events. Source: artifacts/inwild_headtohead.json (paired gbm–cox AUC@90 = –0.107, CI [–0.154, –0.061], excludes 0).

- **Why this matches the literature.** A neutral 34-dataset / 21-model benchmark finds *no method significantly outperforms Cox* on tabular survival [26]; in controlled ablations gradient boosting needs ~600 training samples (~420 events) and the transformer / Cox-likelihood neural models ~1,200 samples (~840 events) to surpass Cox-type linear models [27, 28]. At ~396 events we are firmly in Cox's territory.
- **DeepSurv / DeepHit** are disqualified at this event count — which is also why the GPU sits idle for in-wild: Cox is CPU by design and is the best model.
- **XGBoost-AFT** is the headline ranker for the *abundant* first-weaponization target (c-index 0.598 vs Cox 0.565), where a tree model has enough data to shine.

Mixture-cure is a documented dead-end — do not revive without a Kaplan-Meier plateau. It briefly looked like the only in-wild model with positive IPA on one split, but the rolling-origin backtest overturned it (IPA@180 → –0.27). The cause is theoretical: a cure fraction is only identifiable when the KM curve plateaus above zero, and our ~99.5%-censored target is *administratively* censored, not a cured population. The flat tail is missing follow-up, not immunity.

7. Metrics & Why (Proper Rare-Event Evaluation)

At a <1.5% base rate the standard accuracy/ROC-AUC instincts mislead. The evaluation panel measures the right things under heavy, potentially-informative censoring.

- **IPCW c-index + bootstrap CIs.** The truncated-at-τ variant equals Uno's C [29] under administrative censoring; bootstrap CIs + paired deltas vs Cox replace single point estimates.
- **Time-dependent AUC(t) with IPCW [30, 31].** Discrimination at each fixed horizon, reweighting rather than dropping censored rows.
- **PR-AUC over ROC-AUC.** Under <1% imbalance ROC-AUC looks great while the model is useless on the minority; PR-AUC exposes the true positive yield.
- **Brier / IPA for calibration [32, 33].** IPA = scaled Brier vs the train-KM null — does the *absolute* probability beat the base rate?
- **Decision-curve net-benefit [34].** Measured: the in-wild model beats both treat-all and treat-none across the realistic 0.1–3% threshold band (artifacts/inwild_decision_curve.json) — useful, but modestly.
- **Rolling-origin (walk-forward) backtest.** Each origin trains only on what was knowable then and scores the next period — the honest temporal-validation discipline.

Calibration in practice — the two heads diverge sharply. On the rare in-wild target the Cox reliability plot is nearly degenerate (predictions cluster at the base rate; little spread to calibrate). On the abundant first-weaponization target the XGBoost-AFT plot shows real, if imperfect, calibration across the full 0–1 range.

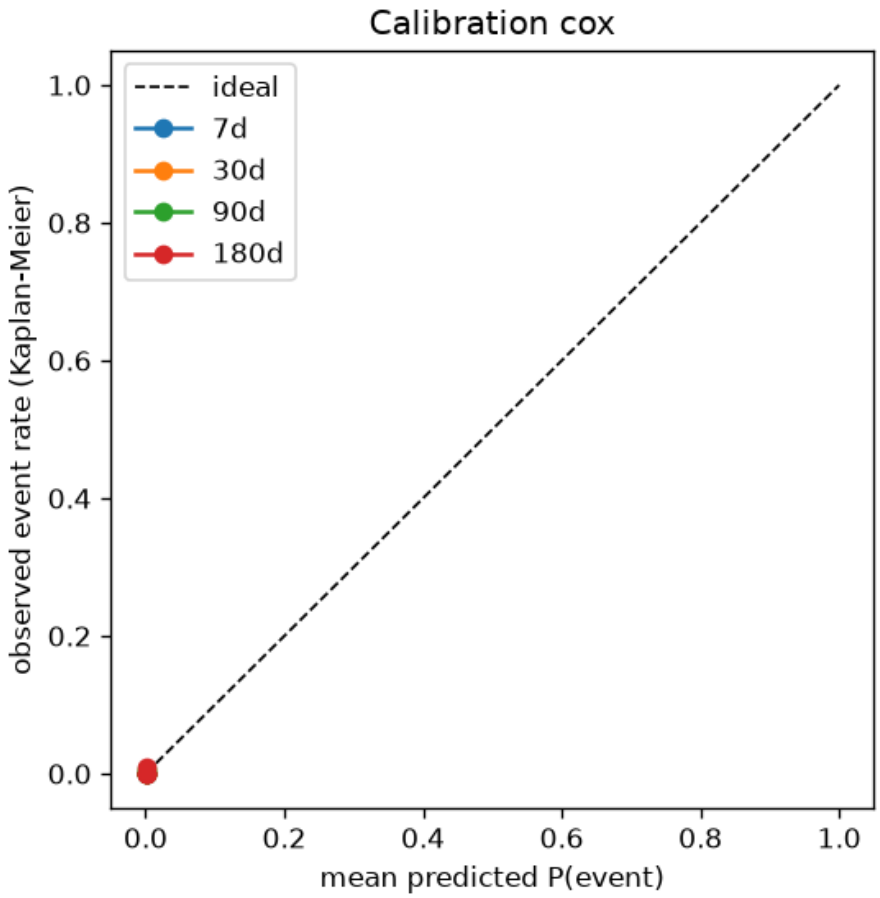

Figure 2. In-wild Cox calibration (70/30 split). Predicted risk barely separates from the base rate — almost nothing to calibrate at ~100 test events. Source: artifacts/reports/inwild_cox_7030/metrics.json.

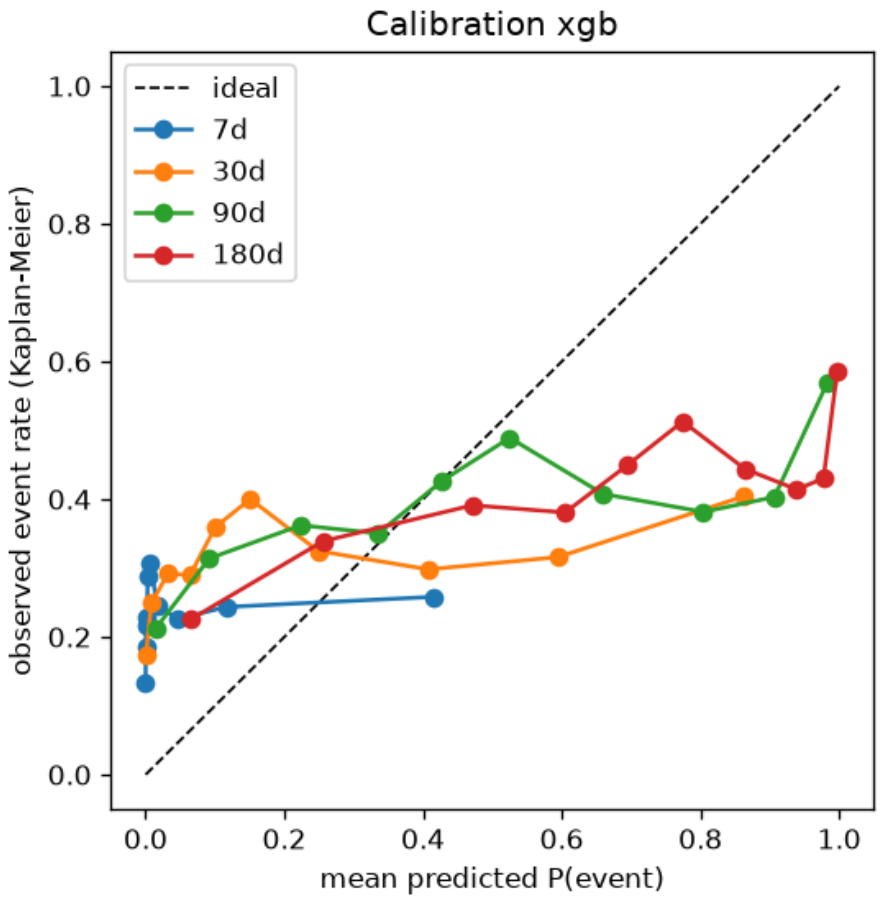

Figure 3. First-weaponization XGBoost-AFT calibration (70/30 split). Predictions span the full range with meaningful reliability — abundance enables calibration. Source: artifacts/reports/firstweap_xgb_7030/metrics.json.

8. The Central Finding — a DATA-Limited Ceiling

The most important conclusion is that the in-wild ceiling is set by **how many confirmed in-wild events exist**, not by model choice. The evidence is a contrast between the two heads at the same 2024-01-01 cutoff:

	In-wild (Cox)	First-weaponization (XGB-AFT)
Events	251	45,947
c-index (IPCW)	0.849	0.598
Ranking quality	strong (AUC@90 ≈ 0.87)	weak (≈ 0.60)
IPA@90 / @180	≈ 0 (slightly negative)	+0.176 / +0.291
Calibration	cannot (nothing to fit)	works
Binding constraint	data scarcity	target noise (PoC logistics)

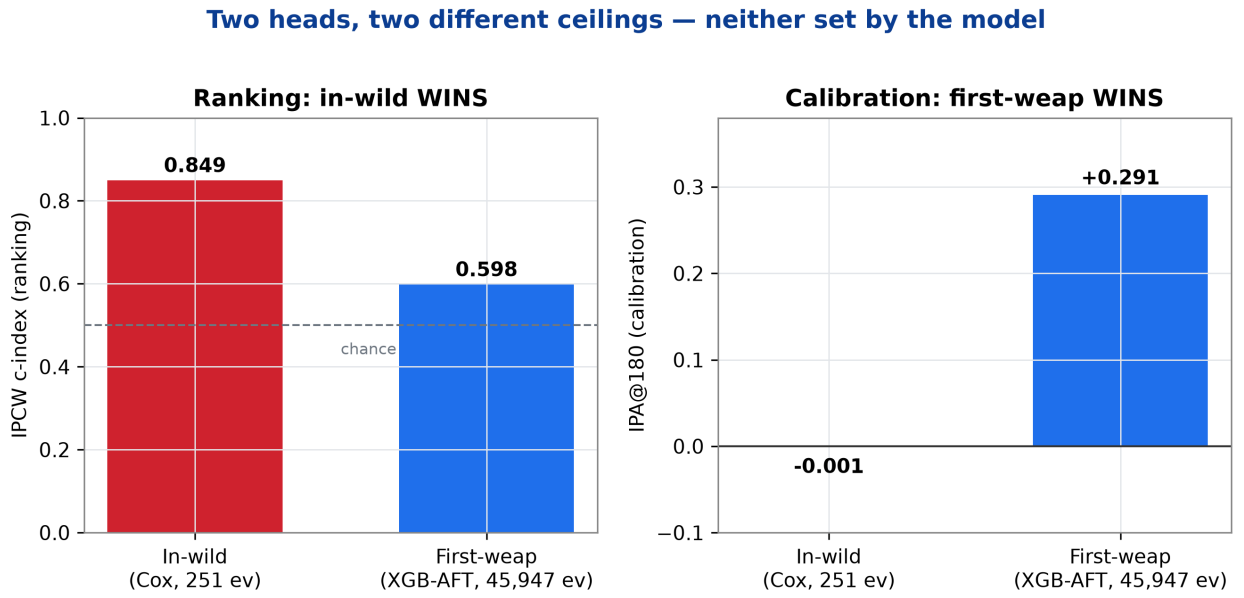

Figure 4. The two-heads diagnostic. The rare in-wild head ranks well but has no calibration headroom; the abundant first-weaponization head calibrates well but ranks weakly. Two different ceilings, neither set by the model. Sources: artifacts/reports/inwild_cox/metrics.json, firstweap_xgb/metrics.json.

Read the two heads together and the diagnosis is unambiguous: where data is abundant *calibration works* — so IPA≈0 on in-wild is not a calibration bug. Where ranking is possible *data is scarce* — so weak first-weap ranking is target noise, not scarcity. Each head is limited by a different thing, and neither limit is the model. Corroborated three ways: every alternative model measured ≤ Cox prospectively; ~9.5 EPV says the remedy is penalization (done), not a bigger model; neutral benchmarks need many hundreds of events (roughly 600–1,200 training samples) before ML or transformer models beat Cox [26, 27].

9. VulnCheck Wiring — Raising the Ceiling

Because the ceiling is data-limited, the highest-leverage action is more / earlier in-wild *labels*. The VulnCheck KEV catalog (~173% larger than CISA KEV, ~27 days earlier [35], using first-reported-*evidence* dates) was fetched and wired end-to-end.

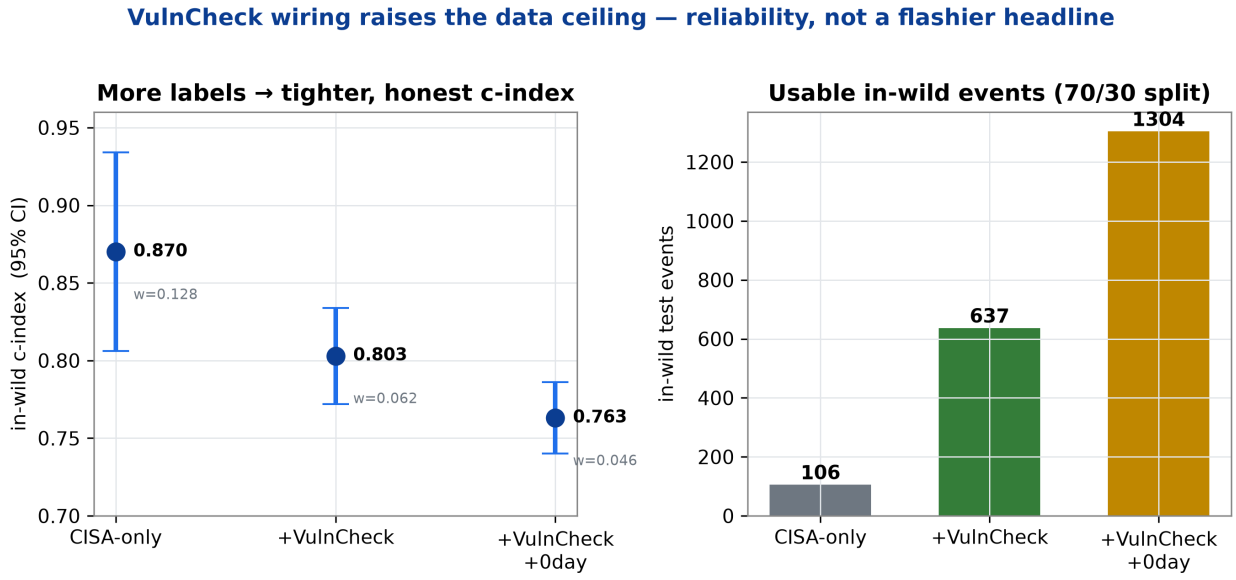

Figure 5. VulnCheck lift on a 70/30 in-wild split. Adding VulnCheck takes test events 106 → 637, roughly halves the c-index CI (width 0.128 → 0.062), and turns IPA@90 positive (3.3e-05 → 0.0070). The point estimate dropping 0.87 → 0.80 is small-sample optimism being corrected. Source: artifacts/reports/inwild_full_wiring.json.

Interpretation: more data buys a more *trustworthy* model, not a flashier headline. The c-index settles honestly near **0.80**, the CI halves, and calibration becomes real (IPA > 0). A tighter interval and genuine calibration are the right kind of win for a rare-event model — reliability, not over-claim.

10. Operational Value — Catch-Rate and Lead Time

For a defender the model's worth is operational: of the CVEs it flags as highest-risk, how many of the eventual in-wild exploitations does it catch, and how much head-start does it give? A short watch window (landmark L) sharply improves both. Source: artifacts/operating_points.json.

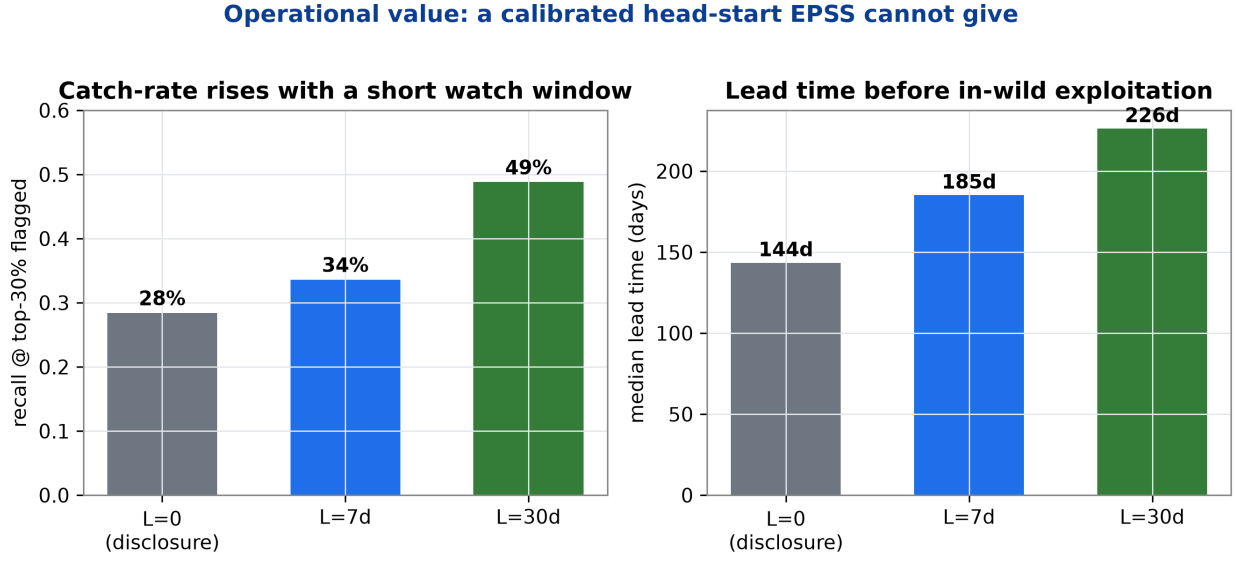

Figure 6. Operating points across landmarks. Flagging the top 30% by risk catches 28% of in-wild exploitations at disclosure (L=0), rising to 49% if you wait 30 days (L=30) — with a median 144–226 day head-start before exploitation. Source: artifacts/operating_points.json.

11. Where the Model Beats EPSS — the Cold-Start Regime

EPSS is near-chance and ~48% missing at disclosure. The defensible value proposition is the **t=0 cold-start in-wild head**: with structured publication-time features the model out-discriminates EPSS-only at exactly the moment a defender has nothing else to go on.

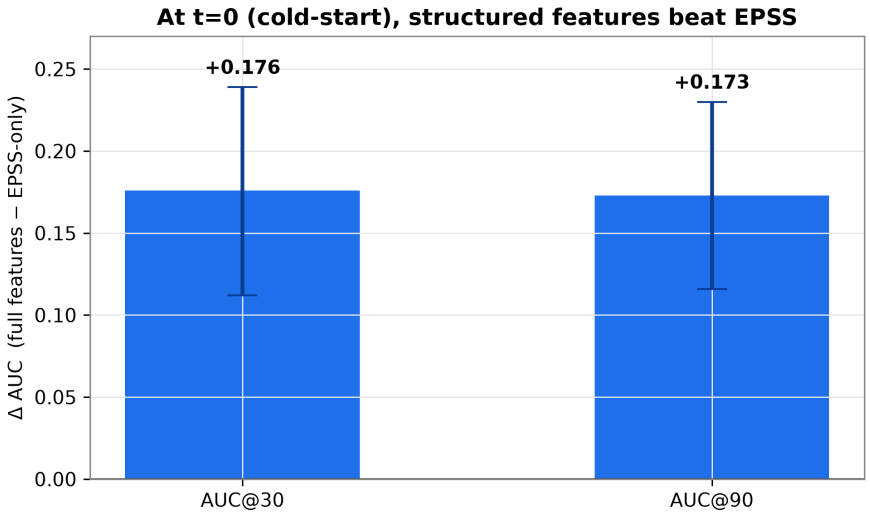

Figure 7. Full features vs EPSS-only at disclosure, paired across 14 walk-forward origins. Structured features add +0.17 AUC at both 30- and 90-day horizons, winning in 93% of origins. Source: artifacts/inwild_epss_ablation.json.

12. Causal Characterization — What *Accelerates* Weaponization

Beyond prediction, a causal layer asks which CVE properties *cause* faster weaponization. It uses an adjusted Cox + stabilized inverse-probability weighting [36] with overlap/positivity diagnostics and the VanderWeele-Ding E-value [37] (unmeasured-confounding robustness). Confounders are pre-treatment common causes only — deliberately not the CVSS components that *define* a treatment. Outcome = time-to-first-weaponization (n=313,847; 147,048 events). Source: artifacts/merged/causal_characterization.json; docs/causal_and_patch_race_2026-06.md.

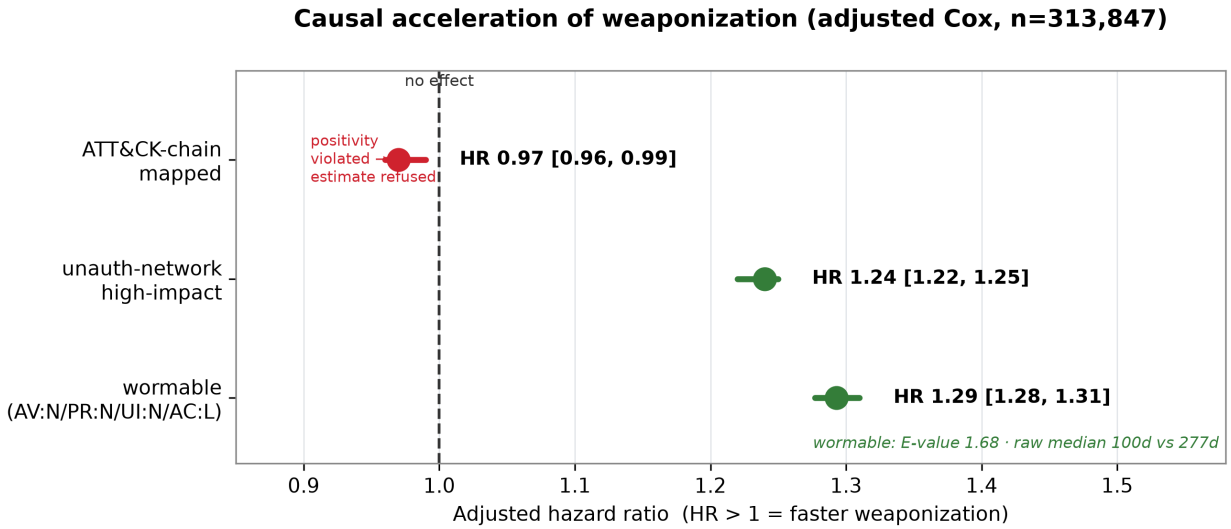

Figure 8. Adjusted hazard ratios. Wormable (network, no-auth, no-UI, low-complexity) and unauthenticated high-impact CVEs are causally weaponized faster (HR 1.29 and 1.24, both robust). ATT&CK-chain; mapping shows no robust effect — positivity is violated, so the framework correctly refuses the estimate. Source: artifacts/merged/causal_characterization.json.

- **Wormable: real causal acceleration.** Adjusted HR 1.29 [1.28, 1.31], IPW 1.41, crude 1.71; raw median time-to-weaponization 100d vs 277d. The effect survives even mediator-inclusive adjustment (HR 1.17) — it lives in 1.17–1.41 across every spec and never crosses 1. E-value 1.68: an unmeasured confounder would need HR≥1.68 with both treatment and outcome to explain it away.
- **ATT&CK-chain-mapped;: no robust effect.** Treated-propensity median 0.94 vs control 0.009 — near-separation. The causal framework refusing this estimate is a feature: it is a null the prior associational log-rank study could not catch.

13. The Patch-vs-Exploit Race

Does the patch or the exploit arrive first? The honest answer reframes the question: **patch-date observability is itself the selector**, so a naive commit-date race model is biased toward cases where the race is already won. Source: artifacts/merged/patch_race.json.

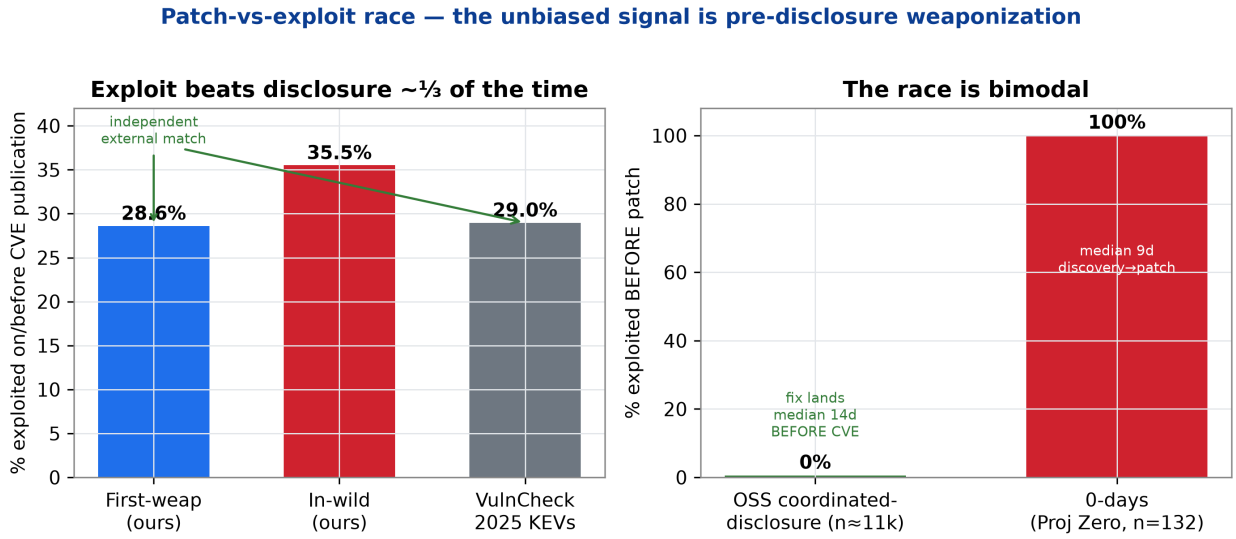

Figure 9. Left: ~■ of weaponizations beat CVE publication — and our 28.6% first-weap rate independently matches VulnCheck's reported 28.96% of 2025 KEVs [38], an external validation of the label pipeline. Right: the race is bimodal — in coordinated-disclosure OSS the fix lands a median 14 days before the CVE (defenders win ~99.5%); 0-days are the mirror (100% exploited before patch). Source: artifacts/merged/patch_race.json.

Methodological lesson: the dangerous exploit-before-patch cases (0-days, vendor-advisory-only, no public commit) are *systematically excluded* from any commit-date cohort. The unbiased race signal is the **pre-disclosure weaponization rate**, computable corpus-wide from labels we already have — no fetch required.

14. Label Completeness — the False-Censoring Gap

The in-wild model right-censors every uncataloged CVE as “never exploited.” How many of those are actually exploited? Using EPSS (FIRST's calibrated P(exploited in 30d) [14, 2], trained on exploitation telemetry) as a semi-independent oracle quantifies the gap. Source: artifacts/merged/label_completeness.json; docs/label_completeness_2026-06.md.

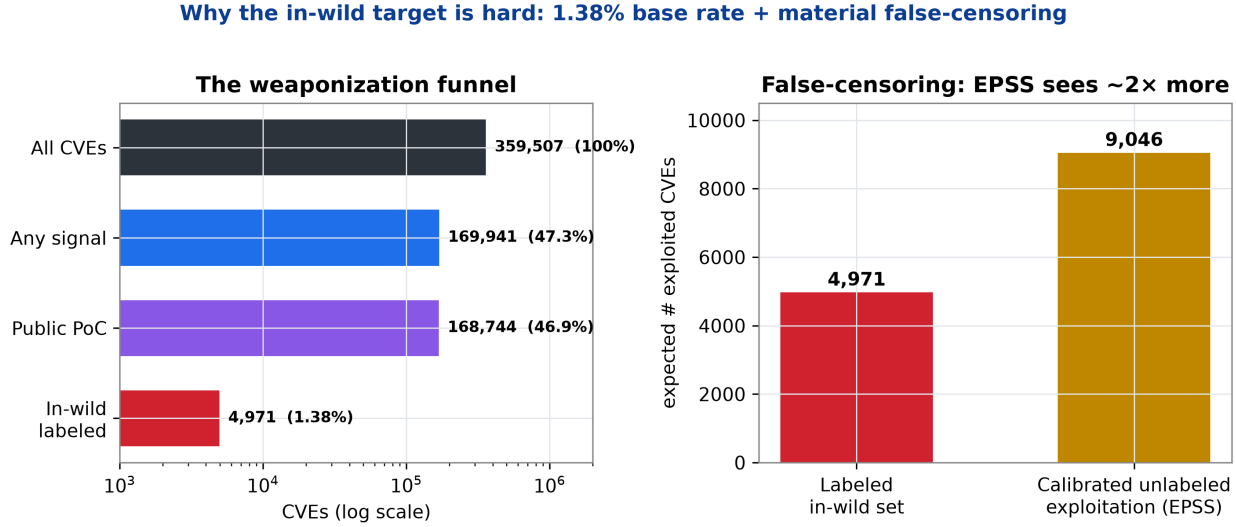

Figure 10. Left: the weaponization funnel — 359,507 CVEs narrow to 1.38% in-wild labeled. Right: a calibrated EPSS expectation of ~9,046 exploited-but-uncataloged CVEs, ~2× the entire labeled in-wild set (97% of the high-EPSS unlabeled

cohort is >1 year old, so this is genuine incompleteness, not catalog lag). Source: artifacts/merged/label_completeness.json.

Honest bottom line: the in-wild ceiling is data-bound, and a *material part* of it is label incompleteness (false-censoring), not just rarity. Closing it requires exploitation **telemetry** (GreyNoise / Shadowserver / VulnCheck ipintel — the paid/keyed modality), not more catalogs and not EPSS-relabeling (which would make the model an EPSS copy). Absent that, the defensible move is to report in-wild results as a **lower bound** and lean on the abundant, well-labeled first-weaponization / PoC→KEV heads.

15. Where to Get More Data (the Roadmap)

Six literature sweeps converged on one meta-finding: the biggest achievable gain is more/earlier in-wild label data. The ranked roadmap:

#	Source / lever	Expected effect	Status
1	VulnCheck KEV (community API)	~3x backtest / 6x 70-30 events, ~27d earlier	DONE
2	Recover self-inflicted label drops	+~1,575 pre-disclosure 0-day events	Free / no API
3	GreyNoise (academic VIP = free)	Earliest mass-exploitation date / volume	Best telemetry; needs application
4	Microsoft MSRC exploited flags	Vendor-confirmed dated exploitation	Wired — measured 0 new (saturated)
5	Shadowserver honeypot feed	Per-CVE in-wild scan attribution	Keyed; scoped to own network
6	AlienVault OTX pulses	Community in-wild indicators	Free API; noisy

Documented dead-ends not to revisit: mixture-cure, RSF/GBM/deep nets, stacked transfer, any recalibration, TabPFN/SurvivalPFN [39, 40] at this scale, and aggregate per-vendor/CWE forecasting (counts measure the reporting apparatus, not attackers).

16. Current State & Next Steps

Current state.

- Test suite: **337 tests collected and green** (deep-model tests are torch-gated).
- In-wild model = penalized Cox on EPSS-enriched, publication-time-safe features, evaluated by the rolling-origin backtest; VulnCheck labels now flow in.
- Causal + patch-race layers committed and externally validated (28.6% ≈ VulnCheck 28.96%).
- Memory stays within the ≤6–8 GB RAM / ≤7 GB VRAM budget on every path.

Next steps (priority order).

- **Recover the ~1,575 self-inflicted negative-duration drops** (pre-disclosure 0-day exploitation) by flooring duration at 0.5d rather than dropping — free, ~50% more in-wild events.
- **Apply for GreyNoise academic VIP** — the one telemetry feed that could close the false-censoring gap (a different modality from catalogs).
- **A higher-signal transition head** (PoC→Metasploit/Nuclei — tooling that genuinely post-dates the PoC), reusing the existing transition machinery.

- **Build the corpus-wide pre-disclosure race model** — unbiased instrument, 28.6% base rate, well-powered, no fetch.

Closing note. The defensible value proposition is the t=0 cold-start in-wild head, where the model beats EPSS by +0.17 AUC at disclosure and adds a calibrated timing curve EPSS structurally cannot give. The project's honesty — documented dead-ends, triple-verified claims, externally-validated labels, and a clear-eyed data-limited verdict — is itself a deliverable.

Appendix A — Module Quick-Reference

Every layer of `src/temporal_exploit/`, the role of each module:

Layer	Module(s)	Role
Spine	cli.py	argparse + 7 *_command() orchestrators
Fetch	fetch/base, cache, gitmine	Connector ABC, ETag cache, git-history mining
	fetch/{kev,epss,nvd,exploitdb,zeroday}	HTTP connectors
	fetch/{poc,nuclei,metasploit}	git-mined connectors
	fetch/{vulncheck,shadowserver}	token/cred-gated in-wild
Merge	merge.py	merge_live() / merge_source() + MERGE_SPECS
Load	loaders.py, schema.py	parquet load + column validation
Labels	labels.py	4 label builders, published = clock origin
Features	features, attack_features, epss_features	publication-time CVSS/CWE/CPE + ATT&CK + EPSS
	nlp_features, text_safety	masked, freshness-gated text (opt-in)
	landmark, poc_features, presence_features	post-pub / transition / snapshot
Split	splits.py	time-based make_time_split()
Models	modeling.py	Cox/RSF/GBM + evaluate + calibration + bootstrap
	baselines, cure, xgb, deep, deephit, survboost	KM / cure / AFT / neural / competing-risk
	competing.py	Aalen-Johansen, cause-specific Cox, transitions
	causal.py	adjusted Cox + IPW + E-value
Eval	evaluate, backtest, simulate	descriptive stats, walk-forward, synthetic DGP
Artifacts	artifacts.py, config.py	manifest + content hashes, paths

Appendix B — Feature Provenance (Leakage Audit)

The provenance table is the institutional memory that prevents leakage creep: 37 feature families across four leakage statuses. Counts: **16 publication-safe** (default), **12 landmark-safe** (opt-in, L=7/30d), **5 transition-safe** (post-PoC), **4 snapshot-leakage** (excluded). Full detail in `artifacts/feature_provenance.csv`.

leakage_status	# families	default?	guard
publication_time_safe	16	YES	knowable at disclosure
landmark_safe	12	opt-in	requires restart_clock(L)
transition_safe_post_poc	5	opt-in	post-PoC clock origin
snapshot_leakage	4	NEVER	kept separate for audit only

References

Each source below was verified against its DOI, arXiv identifier, or official venue page during preparation of this report (2026-06); none is auto-generated or unverified. Citations are numbered in order of first appearance. Method papers ground the survival / evaluation / causal machinery; domain references ground the exploitation-timing and EPSS / KEV claims.

[1] J. Jacobs, S. Romanosky, B. Edwards, M. Roytman, and I. Adjerid. "Exploit Prediction Scoring System (EPSS)." *Digital Threats: Research and Practice*, 2(3):Article 20, 2021. doi:10.1145/3436242.

[2] FIRST.org EPSS Special Interest Group. "The EPSS Model." Forum of Incident Response and Security Teams. https://www.first.org/epss/model (accessed 2026).

[3] E. L. Kaplan and P. Meier. "Nonparametric Estimation from Incomplete Observations." *Journal of the American Statistical Association*, 53(282):457–481, 1958. doi:10.1080/01621459.1958.10501452.

[4] D. R. Cox. "Regression Models and Life-Tables." *Journal of the Royal Statistical Society: Series B (Methodological)*, 34(2):187–202, 1972. doi:10.1111/j.2517-6161.1972.tb00899.x.

[5] N. Simon, J. Friedman, T. Hastie, and R. Tibshirani. "Regularization Paths for Cox's Proportional Hazards Model via Coordinate Descent." *Journal of Statistical Software*, 39(5):1–13, 2011. doi:10.18637/jss.v039.i05.

[6] H. Ishwaran, U. B. Kogalur, E. H. Blackstone, and M. S. Lauer. "Random Survival Forests." *The Annals of Applied Statistics*, 2(3):841–860, 2008. doi:10.1214/08-AOAS169.

[7] T. Chen and C. Guestrin. "XGBoost: A Scalable Tree Boosting System." In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD 16)*, pp. 785–794, 2016. doi:10.1145/2939672.2939785.

[8] A. Barnwal, H. Cho, and T. Hocking. "Survival Regression with Accelerated Failure Time Model in XGBoost." *Journal of Computational and Graphical Statistics*, 31(4):1292–1302, 2022. doi:10.1080/10618600.2022.2067548.

[9] V. T. Farewell. "The Use of Mixture Models for the Analysis of Survival Data with Long-Term Survivors." *Biometrics*, 38(4):1041–1046, 1982. doi:10.2307/2529885.

[10] J. L. Katzman, U. Shaham, A. Cloninger, J. Bates, T. Jiang, and Y. Kluger. "DeepSurv: Personalized Treatment Recommender System Using a Cox Proportional Hazards Deep Neural Network." *BMC Medical Research Methodology*, 18:Article 24, 2018. doi:10.1186/s12874-018-0482-1.

[11] C. Lee, W. R. Zame, J. Yoon, and M. van der Schaar. "DeepHit: A Deep Learning Approach to Survival Analysis with Competing Risks." In *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1):2314–2321, 2018. doi:10.1609/aaai.v32i1.11842.

[12] J. P. Fine and R. J. Gray. "A Proportional Hazards Model for the Subdistribution of a Competing Risk." *Journal of the American Statistical Association*, 94(446):496–509, 1999. doi:10.1080/01621459.1999.10474144.

[13] O. O. Aalen and S. Johansen. "An Empirical Transition Matrix for Non-Homogeneous Markov Chains Based on Censored Observations." *Scandinavian Journal of Statistics*, 5(3):141–150, 1978. JSTOR:4615704.

[14] J. Jacobs, S. Romanosky, O. Suciu, B. Edwards, and A. Sarabi. "Enhancing Vulnerability Prioritization: Data-Driven Exploit Predictions with Community-Driven Insights." In *2023 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pp. 194–206, 2023. arXiv:2302.14172.

[15] C. Sabottke, O. Suciu, and T. Dumitras. "Vulnerability Disclosure in the Age of Social Media: Exploiting Twitter for Predicting Real-World Exploits." In *24th USENIX Security Symposium (USENIX Security 15)*, 2015.

[16] L. Allodi and F. Massacci. "Comparing Vulnerability Severity and Exploits Using Case-Control Studies." *ACM Transactions on Information and System Security (TISSEC)*, 17(1):Article 1, 2014. doi:10.1145/2630069.

[17] A. D. Householder, J. Chrabaszcz, T. Novelly, D. Warren, and J. M. Spring. "Historical Analysis of Exploit Availability Timelines." In *13th USENIX Workshop on Cyber Security Experimentation and Test (CSET 20)*, 2020.

[18] K. A. Farris, A. Shah, S. Jajodia, R. Ganesan, and G. Cybenko. "VULCON: A System for Vulnerability Prioritization, Mitigation, and Management." *ACM Transactions on Privacy and Security (TOPS)*, 21(4):Article 16, 2018.

doi:10.1145/3196884.

[19] Cybersecurity and Infrastructure Security Agency (CISA). "Known Exploited Vulnerabilities Catalog." https://www.cisa.gov/known-exploited-vulnerabilities-catalog (accessed 2026).

[20] S. Paul. "How Good Is EPSS, Really? A Five-Year Empirical Evaluation Correcting for Look-Ahead Bias." IEEE DataPort, 2026. doi:10.21227/bhhs-0994. (Non-peer-reviewed dataset/report.)

[21] C. Davidson-Pilon. "lifelines: Survival Analysis in Python." *Journal of Open Source Software*, 4(40):1317, 2019. doi:10.21105/joss.01317.

[22] P. Peduzzi, J. Concato, A. R. Feinstein, and T. R. Holford. "Importance of Events per Independent Variable in Proportional Hazards Regression Analysis. II. Accuracy and Precision of Regression Estimates." *Journal of Clinical Epidemiology*, 48(12):1503–1510, 1995.

[23] E. Vittinghoff and C. E. McCulloch. "Relaxing the Rule of Ten Events per Variable in Logistic and Cox Regression." *American Journal of Epidemiology*, 165(6):710–718, 2007. doi:10.1093/aje/kwk052.

[24] T. Hothorn, P. Buhlmann, S. Dudoit, A. Molinaro, and M. J. van der Laan. "Survival Ensembles." *Biostatistics*, 7(3):355–373, 2006. doi:10.1093/biostatistics/kxj011.

[25] S. Polsterl. "scikit-survival: A Library for Time-to-Event Analysis Built on Top of scikit-learn." *Journal of Machine Learning Research*, 21(212):1–6, 2020. https://jmlr.org/papers/v21/20-729.html.

[26] L. Burk, J. Zobolas, B. Bischl, A. Bender, M. N. Wright, and R. Sonabend. "A Large-Scale Neutral Comparison Study of Survival Models on Low-Dimensional Data." *Bioinformatics*, 42(5), 2026. arXiv:2406.04098; doi:10.1093/bioinformatics/btag186.

[27] I. Rossi, F. Sartori, C. Rollo, G. Birolo, P. Fariselli, and T. Sanavia. "Beyond Cox Models: Assessing the Performance of Machine-Learning Methods in Non-Proportional Hazards and Non-Linear Survival Analysis." arXiv:2504.17568, 2025.

[28] Z. Wang and J. Sun. "SurvTRACE: Transformers for Survival Analysis with Competing Events." In *Proceedings of the 13th ACM International Conference on Bioinformatics, Computational Biology and Health Informatics (ACM-BCB)*, 2022. arXiv:2110.00855.

[29] H. Uno, T. Cai, M. J. Pencina, R. B. D'Agostino, and L. J. Wei. "On the C-Statistics for Evaluating Overall Adequacy of Risk Prediction Procedures with Censored Survival Data." *Statistics in Medicine*, 30(10):1105–1117, 2011. doi:10.1002/sim.4154.

[30] P. Blanche, J.-F. Dartigues, and H. Jacqmin-Gadda. "Estimating and Comparing Time-Dependent Areas Under Receiver Operating Characteristic Curves for Censored Event Times with Competing Risks." *Statistics in Medicine*, 32(30):5381–5397, 2013. doi:10.1002/sim.5958.

[31] P. J. Heagerty and Y. Zheng. "Survival Model Predictive Accuracy and ROC Curves." *Biometrics*, 61(1):92–105, 2005. doi:10.1111/j.0006-341X.2005.030814.x.

[32] E. Graf, C. Schmoor, W. Sauerbrei, and M. Schumacher. "Assessment and Comparison of Prognostic Classification Schemes for Survival Data." *Statistics in Medicine*, 18(17–18):2529–2545, 1999.

[33] M. W. Kattan and T. A. Gerds. "The Index of Prediction Accuracy: An Intuitive Measure Useful for Evaluating Risk Prediction Models." *Diagnostic and Prognostic Research*, 2:Article 7, 2018. doi:10.1186/s41512-018-0029-2.

[34] A. J. Vickers and E. B. Elkin. "Decision Curve Analysis: A Novel Method for Evaluating Prediction Models." *Medical Decision Making*, 26(6):565–574, 2006. doi:10.1177/0272989X06295361.

[35] VulnCheck, Inc. "VulnCheck KEV Surges to Track More than 3,600 Known Exploited Vulnerabilities." Press release, May 2025. https://www.vulncheck.com/press/vulncheck-kev-10000.

[36] J. M. Robins, M. A. Hernan, and B. Brumback. "Marginal Structural Models and Causal Inference in Epidemiology." *Epidemiology*, 11(5):550–560, 2000. doi:10.1097/00001648-200009000-00011.

[37] T. J. VanderWeele and P. Ding. "Sensitivity Analysis in Observational Research: Introducing the E-Value." *Annals of Internal Medicine*, 167(4):268–274, 2017. doi:10.7326/M16-2607.

[38] P. Garrity. "State of Exploitation: A Look at 2025 Vulnerability Exploitation." VulnCheck, Jan. 2026. https://www.vulncheck.com/blog/state-of-exploitation-2026.

[39] N. Hollmann, S. Muller, K. Eggensperger, and F. Hutter. "TabPFN: A Transformer That Solves Small Tabular Classification Problems in a Second." In *International Conference on Learning Representations (ICLR)*, 2023. (Extended as "Accurate Predictions on Small Data with a Tabular Foundation Model," *Nature*, 637:319–326, 2025, doi:10.1038/s41586-024-08328-6.)

[40] S.-A. Qi, V. Balazadeh, M. Cooper, R. Greiner, and R. G. Krishnan. "SurvivalPFN: Amortizing Survival Prediction via In-Context Bayesian Inference." arXiv:2605.15488, 2026.